Also note that $d(o, t_o) \leq d(o, t_x)$ for any x. Hence

$$\begin{aligned} R &\leq \sum_{o \in O} \sum_{x \in C_o{}^*} (d(x,o)^2 + d(o,t_x)^2) \\ &= \sum_{x \in \mathcal{C}} (d(x,o_x)^2 + d(o_x,t_x)^2) \end{aligned}$$

Using triangle inequality we know that $d(o_x, t_x) \leq d(o_x, x) + d(x, t_x)$. Substituting above and expanding we get that

$$R \leq 2\Phi(O) + \Phi(T) + 2\sum_{x \in \mathcal{C}} d(x,o_x)d(x,t_x) \tag{4.4}$$

The last term in the above equation can be bounded using Cauchy-Schwarz inequality as $\sum_{x \in \mathcal{C}} d(x,o_x)d(x,t_x) \leq \sqrt{\Phi(O)}\sqrt{\Phi(S)}$. So we have that $R \leq 2\Phi(O) + \Phi(T) + 2\sqrt{\Phi(O)}\sqrt{\Phi(S)}$. Substituting this in Equation 4.3 and solving we get the desired result that $\Phi(T) \leq 25\Phi(O)$. Combining this with Lemma 6 proves Theorem 5.

A natural generalization of Algorithm LOCAL SEARCH is to swap more than one centers at each step. This could potentially lead to a much better local optimum. This multi-swap scheme was analyzed by [47] and using a similar analysis as above one can show the following

Theorem 8. *Let S be the final set of centers by the local search algorithm which swaps upto p centers at a time. Then we have that $\Phi(S) \leq 2(3 + \frac{2}{p})^2\mathrm{OPT}$, where* OPT *is the cost of the optimal k-means solution.*

For the case of k-median the same algorithm and analysis gives [16]

Theorem 9. *Let S be the final set of centers by the local search algorithm which swaps upto p centers at a time. Then we have that $\Phi(S) \leq (3 + \frac{2}{p})\mathrm{OPT}$, where* OPT *is the cost of the optimal k-median solution.*

This approximation factor for k-median has recently been improved to $(1 + \sqrt{3} + \epsilon)$ [50]. For the case of k-means in Euclidean space [48] give an algorithm which achieves a $(1 + \epsilon)$ approximation to the k-means objective for any constant $\epsilon > 0$. However the runtime of the algorithm depends exponentially in k and hence it is only suitable for small instances.

5 Clustering of *stable* instances

In this part of the chapter we delve into some of the more modern research in the theory of clustering. In recent past there has been an increasing interest in designing clustering algorithms that enjoy strong theoretical guarantees on non-worst case instance. This is of significant interest for two reasons: a) From a theoretical point of view, this helps us understand and characterize the class of problems for which one can get optimal or close

to optimal guarantees, b) From a practical point of view, real world instances often have additional structure that could be exploited to get better performance. Compared to worst case analysis, the main challenge here is to formalize well motivated and interesting additional structures of clustering instances under which good algorithms exist. In this section we present two popular interesting notions.

5.1 ϵ-separability

This notion of stability was proposed by Ostrovsky et al.[57]. Given an instance of k-means clustering, let $\mathrm{OPT}(k)$ denote the cost of the optimal k-means solution. We can also decompose $\mathrm{OPT}(k)$ as $\mathrm{OPT} = \sum_{i=1}^{k} \mathrm{OPT}_i$, where OPT_i denotes the 1-means cost of cluster C_i, i.e., $\sum_{x \in C_i} d(x, c_i)^2$. Such an instance is called *ϵ-separable* if it satisfies $\mathrm{OPT}(k-1) > \frac{1}{\epsilon^2}\mathrm{OPT}(k)$.

The definition is motivated by the following issue: when approaching a clustering problem, one typically has to decide how many clusters one wants to partition the data in, i.e., the value of k. If the k-means objective is the underlying criteria being used to judge the quality of a clustering, and the optimal $(k-1)$-means clustering is comparable to the optimal k-means clustering, then one can in principle also use $(k-1)$ clusters to describe the data set. In fact this particular method is a very popular heuristic to find out the number of hidden clusters in the data set. In other words choose the value of k at which there is a significant increase in the k-means cost when going from k to $k-1$. As an illustrative example consider the case of a mixture of k spherical unit variance Gaussians in d dimensions whose pair wise means are separated by a distance $D >> 1$. Given n points from each Gaussian, the optimal k-means cost with high probability is nkd. On the other hand, if we try to cluster this data using $(k-1)$ clusters, the optimal cost will now become $n(k-1)d + n(D^2 + d)$. Hence, taking the ratio of the two costs, this instance will be ϵ-separable for $\frac{1}{\epsilon^2} = \frac{(k-1)d + D^2 + d}{kd} = 1 + \frac{D^2}{kd}$), so $\epsilon = (1 + \frac{D^2}{kd})^{-1/2}$. Hence, if $D \gg \sqrt{kd}$, then the instance will be highly separable (the separability parameter ϵ will be $o(1)$).

It was shown by Ostrovsky et al. [57] that one can design much better approximation algorithms for ϵ-separable instances.

Theorem 10 ([57]). *There is a polynomial time algorithm which given any ϵ-separable 2-means instance returns a clustering of cost at most $\frac{OPT}{1-\rho}$ with probability at least $1 - O(\rho)$ where $c_2\epsilon^2 \leq \rho \leq c_1\epsilon^2$ for some constants $c_1, c_2 > 0$.*

Theorem 11 ([57]). *There is a polynomial time algorithm which given any ϵ-separable k-means instance a clustering of cost at most $\frac{\mathrm{OPT}}{1-\rho}$ with probability $1 - O((\rho)^{1/4})$ where $c_2\epsilon^2 \leq \rho \leq c_1\epsilon^2$ for some constants $c_1, c_2 > 0$.*

5.2 Proof Sketch and Intuition for Theorem 10

Notice that the above algorithm does not need to know the value of ϵ from the separability of the instance. Define r_i to be the radius of cluster C_i in the optimal k-means clustering,

i.e., ${r_i}^2 = \frac{\text{OPT}_i}{|C_i|}$. The main observation is that under the ϵ-separability condition, the optimal k-means clustering is "spread out". In other words, the radius of any cluster is much smaller than the inter cluster distances. This can be formulated in the following lemma

Lemma 12. $\forall i, j,\ d(c_i, c_j)^2 \geq \frac{1-\epsilon^2}{\epsilon^2} max({r_i}^2, {r_j}^2)$.

Proof. Given an ϵ-separable instance of k-means, consider any two clusters C_i and C_j in the optimal clustering with centers c_i and c_j respectively. Consider the $(k-1)$ clustering obtained by deleting c_j and assigning all the points in C_j to C_i. By ϵ-separability, the cost of this new clustering must be at least $\frac{\text{OPT}}{\epsilon^2}$. However the increase in the cost will be exactly $|C_j| d(c_i, c_j)^2$. This follows from the simple observation stated in Fact 2. Hence we have that $|C_j| d(c_i, c_j)^2 > (\frac{1}{\epsilon^2} - 1)\text{OPT}$. This gives us that ${r_j}^2 = \frac{\text{OPT}}{|C_j|} \leq \frac{\epsilon^2}{1-\epsilon^2} d(c_i, c_j)^2$. Similarly, if we delete c_i and assign all the points in C_i to C_j we get that ${r_i}^2 \leq \frac{\epsilon^2}{1-\epsilon^2} d(c_i, c_j)^2$. □

When dealing with the two means problem, if one could find two initial candidate center points which are close to the corresponding optimal centers, then we could hope to run a Lloyd's type step and improve the solution quality. In particular if we could find $\bar{c_1}$ and $\bar{c_2}$ such that $d(c_1, \bar{c_1})^2 \leq \alpha {r_1}^2$ and $d(c_2, \bar{c_2})^2 \leq \alpha {r_2}^2$, then we know from Fact 2 that using these center points will give us a $(1+\alpha)$ approximation to OPT. Lemma 12 suggests the following approach: pick data points x, y with probability proportional to $d(x, y)^2$. We will show that this will lead to seed points $\hat{c_1}$ and $\hat{c_2}$ not too far from the optimal centers. Applying a Lloyd type reseeding step will then lead us to the final centers which will be much closer to the optimal centers. We start by defining the core of a cluster.

Definition 3 (Core of a cluster)**.** *Let $\rho < 1$ be a constant. We define $X_i = \{x \in C_i : d(x, c_i)^2 \leq \frac{{r_i}^2}{\rho}\}$. We call X_i as the core of the cluster C_i.*

We next show that if we pick initial seeds $\{\hat{c_1}, \hat{c_2}\} = \{x, y\}$ with probability proportional to $d(x, y)^2$ then with high probability the points lie within the core of different clusters.

Lemma 13. *For sufficiently small ϵ and $\rho = \frac{100\epsilon^2}{1-\epsilon^2}$, we have $Pr[\{\hat{c_1}, \hat{c_2}\} \cap X_1 \neq \emptyset$ and $\{x, y\} \cap X_2 \neq \emptyset] = 1 - O(\rho)$.*

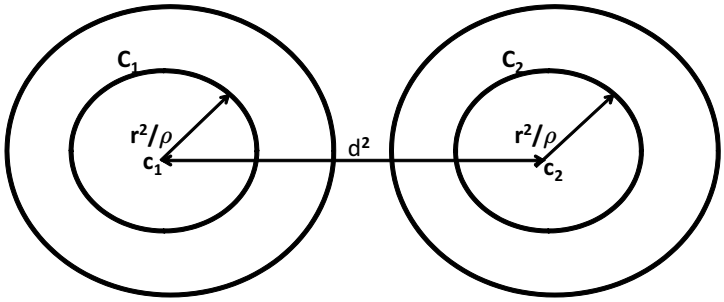

Figure 3: An ϵ-separable 2-means instance

Proof Sketch. For simplicity assume that the sizes of the two clusters is the same, i.e., $|C_i| = |C_j| = n/2$. In this case, we have ${r_1}^2 = {r_2}^2 = \frac{2OPT}{n} = r^2$. Also, let $d^2(c_1, c_2) = d^2$.

From ε-separability, we know that $d^2 > \frac{1-\epsilon^2}{\epsilon^2} r^2$. Also, from the definition of the core, we know that at least $(1-\rho)$ fraction of the mass of each cluster lies within the core. Hence, the clustering instance looks like the one showed in Figure 3. Let $A = \sum_{x \in X_1, y \in X_2} d(x,y)^2$ and $B = \sum_{x,y \subset \mathcal{C}} d(x,y)^2$. Then the probability of the event is exactly $\frac{A}{B}$. Let's analyze quantity B first. The proof goes by arguing that the pairwise distances between X_1 and X_2 will dominate B. This is because of Lemma 12 which says that d^2 is much greater than r^2, the average radius of a cluster. More formally, From Corollary 3 and from Fact 4 we can get that $B = n{\Delta_1}^2(\mathcal{C}) = n{\Delta_2}^2(\mathcal{C}) + n^2/4d^2$. In addition ϵ-separability tells us that ${\Delta_1}^2(\mathcal{C}) > 1/\epsilon^2 {\Delta_2}^2(\mathcal{C})$. Hence we get that $B \leq \frac{n^2}{4(1-\epsilon^2)} d^2$.

Let's analyze $A = \sum_{x \in X_1, y \in X_2} d(x,y)^2$. From triangle inequality, we have that for any $x \in X_1$, $y \in X_2$, $d^2(x,y) \geq (d - 2r/\sqrt{\rho})^2$. Hence $A \geq \frac{1}{4}(1-\rho)^2 n^2 (d - 2r/\sqrt{\rho})^2$. Substituting these bounds and using the fact that $\rho O(\epsilon^2)$, gives us that $A/B \geq (1 - O(\rho))$.

□

Using these initial seeds we now show that a single step of a Lloyd's type method can yield good a solution. Define $r = d(\hat{c_1}, \hat{c_2})/3$. Define $\bar{c_1}$ as the mean of the points in $B(\hat{c_1}, r)$ and $\bar{c_2}$ as the mean of the points in $B(\hat{c_2}, r)$. Notice that instead of taking the mean of the Voronoi partition corresponding to $\hat{c_1}$ and $\hat{c_2}$, we take the mean of the points within a small radius of the given seeds.

Lemma 14. *Given $\hat{c_1} \in X_1$ and $\hat{c_2} \in X_2$, the clustering obtained using $\bar{c_1}$ and $\bar{c_2}$ as centers has 2-means cost at most $\frac{OPT}{1-\rho}$.*

Proof. We will first show that $X_1 \subseteq B(\hat{c_1}, r) \subseteq C_1$. Using Lemma 12 we know that $d(\hat{c_1}, c_1) \leq \frac{\epsilon}{\rho(1-\epsilon^2)} d(c_1, c_2) \leq d(c_1, c_2)/10$ for sufficiently small ϵ. Similarly $d(\hat{c_2}, c_2) \leq d(c_1, c_2)/10$. Hence we get that $4/5 \leq r \leq 6/5$. So for any $z \in B(\hat{c_1}, r)$, $d(z, c_1) \leq d(c_1, c_2)/2$. Hence $z \in C_1$. Also for any $z \in X_1$, $d(z, \hat{c_1}) \leq 2\frac{{r_1}^2}{\rho} \leq r$. Similarly one can show that $X_2 \subseteq B(\hat{c_2}, r) \subseteq C_2$. Now applying Fact 4 we can claim that $d(\bar{c_1}, c_1) \leq \frac{\rho}{1-\rho} {r_1}^2$ and $d(\bar{c_2}, c_2) \leq \frac{\rho}{1-\rho} {r_2}^2$. So using $\bar{c_1}$ and $\bar{c_2}$ as centers we get a clustering of cost at most $\text{OPT} + \frac{\rho}{1-\rho}\text{OPT} = \frac{\text{OPT}}{1-\rho}$. □

Summarizing the discussion above, we have the following simple algorithm for the 2-means problem.

Algorithm 2-MEANS

1. **Seeding:** Choose initial seeds x, y with probability proportional to $d(x,y)^2$.
2. Given seeds $\hat{c_1}, \hat{c_2}$, let $r = d(\hat{c_1}, \hat{c_2})/3$. Define $\bar{c_1} = mean(B(\hat{c_1}, r))$ and $\bar{c_2} = mean(B(\hat{c_2}, r))$.
3. **Output:** $\bar{c_1}$ and $\bar{c_2}$ as the cluster centers.

5.3 Proof Sketch and Intuition for Theorem 11

In order to generalize the above argument to the case of k clusters, one could follow a similar approach and start with k initial seed centers. Again we start by choosing x, y with probability proportional to $d(x,y)^2$. After choosing a set of U of points, we choose the next point z with probability proportional to $min_{\hat{c}_i \in U} d(z, \hat{c}_i)^2$. Using a similar analysis as in Lemma 13 one can show that if we pick k seeds then with probability $(1 - O(\rho))^k$ they will lie with the cores of different clusters. However this probability of success is exponentially small in k and is not good for our purpose. The approach taken in [57] is to sample a larger set of points and argue that with high probability it is going to contain k seed points from the "outer" cores of different clusters. Here we define outer core of a cluster as $X_i^{out} = \{x \in C_i : d(x, c_i)^2 \leq \frac{{r_i}^2}{\rho^3}\}$ – so this notion is similar to the core notion for $k = 2$ except that the radius of the core is bigger by a factor of $1/(\rho)$ than before. We would like to again point out a similar seeding procedure as the one described above is used in the k-means++ algorithm [15](See Section 2). One can show that using k seed centers in this way gives an $O(\log(k))$-approximation to the k-means objective in the worst case.

Lemma 15 ([57]). *Let $N = \frac{2k}{1-5\rho} + \frac{2\ln(2/\delta)}{(1-5\rho)^2}$, where $\rho = \sqrt{\epsilon}$. If we sample N points using the sampling procedure then $Pr[\forall j = 1 \cdots k$, there exists some $\hat{x}_i \in {X_j}^{out}] \geq 1 - \delta$*

Since we sample more than k points in the first step, one needs to extract k good seed points out of this set before running the Lloyd step. This is achieved by the following greedy procedure:

Algorithm GREEDY DELETION PROCEDURE

1. Let S denote the current set of candidate centers. Let $\Phi(S)$ denote the k-means cost of the Voronoi partition using S. Similarly, for $x \in S$ denote $\Phi(S_x)$ be the k-means cost of the Voronoi partition using $S \setminus \{x\}$.
2. **While** $|S| > k$,
   - remove a point x from S, such that $\Phi(S_x) - \Phi(S)$ is minimum.
   - For every remaining point $x \in S$, let $R(x)$ denote the Voronoi set corresponding to x. Replace x by $mean(R(x))$.
   - **Output:** S.

At the end of the greedy procedure we have the following guarantee

Lemma 16. *For every optimal center c_i, there is a point $\hat{c}_i \in S$, such that $d(c_i, \hat{c}_i) \leq \frac{D_i}{10}$. Here $D_i = min_{j \neq i}\, d(c_i, c_j)$.*

Using the above lemma and applying the same Lloyd step as in the 2-means problem we get a set of k good final centers. These centers have the property that for each i, $d(c_i, \bar{c}_i) \leq \frac{\rho}{1-\rho} {r_i}^2$. Putting the above argument formally we get the desired result.

5.4 Approximation stability

In [20] Balcan et al. introduce and analyze a class of approximation stable instances for which they provide polynomial time algorithms for finding accurate clustering. The starting point of this work, is that for many problems of interest to machine learning, such as as clustering proteins by function, images by subject, or documents by topic, there is some unknown correct target clustering. In such cases the implicit hope when pursuing an objective based clustering approach (k-means or k-median) is that approximately optimizing the objective function will in fact produce a clustering of low clustering error, i.e. a clustering that is point wise close to the target clustering. Balcan et al. have shown that by making this implicit assumption explicit, one can efficiently compute a low-error clustering even in cases when the approximation problem of the objective function is NP-complete! This is quite interesting since it shows that by exploiting the properties of the problem at hand one can solve the desired problem and bypass worst case hardness results. A similar stability assumption, regarding additive approximations, was presented in [54]. The work of [54] studied sufficient conditions under which the stability assumption holds true.

Formally, the *approximation stability* notion is defined as follows:

Definition 4 (($(1+\alpha,\epsilon)$-approximation-stability)). *Let X be a set of n points residing in a metric space $\mathcal{M}$. Given an objective function Φ (such as k-median, k-means, or min-sum), we say that instance $(\mathcal{M}, X)$* satisfies $(1+\alpha,\epsilon)$-approximation-stability for Φ *if all clusterings $\mathcal{C}$ with $\Phi(\mathcal{C}) \leq (1+\alpha)\cdot \mathrm{OPT}_\Phi$ are ϵ-close to the target clustering $\mathcal{C}_T$ for $(\mathcal{M}, S)$.*

Here the term "target" clustering refers to the ground truth clustering of X which one is trying to approximate. It is also important to clarify what we mean by an ϵ-close clustering. Given two k clusterings $\mathcal{C}$ and $\mathcal{C}^*$ of n points, the distance between them is measured as $dist(\mathcal{C},\mathcal{C}^*) = \min_{\sigma\in S_k} \frac{1}{n}\sum_{i=1}^{k} |C_i \setminus {C^*}_{\sigma(i)}|$. We say that $\mathcal{C}$ is ϵ-close to $\mathcal{C}^*$ if the distance between them is at most ϵ. Interestingly, this approximation stability condition implies a lot of structure about the problem instance which could be exploited algorithmically. For example, we can show the following.

Theorem 17. *[[20]] If the given instance $(\mathcal{M}, S)$ satisfies $(1+\alpha,\epsilon)$-approximation-stability for the k-median or the k-means objective, then we can efficiently produce a clustering that is $O(\epsilon+\epsilon/\alpha)$-close to the target clustering $\mathcal{C}_T$.*

Notice that the above theorem is valid even for values of α for which getting a $(1+\alpha)$-approximation to k-median and k-means is NP-hard! In a recent paper, [4] show that running the kmeans++ algorithm for approximation stable instances of k-means gives a constant factor approximation with probability $\Omega(\frac{1}{k})$. In the following we will provide a sketch of the proof of Theorem 17 for k-means clustering.

5.5 Proof Sketch and Intuition for Theorem 17

Let $C_1, C_2, \cdots, C_k$ be an optimal k-means clustering of $\mathcal{C}$. Let $c_1, c_2, \cdots, c_k$ be the corresponding cluster centers. For any point $x \in \mathcal{C}$, let $w(x)$ be the distance of x to its cluster center. Similarly let $w_2(x)$ be the distance of x to the second closest center. The value of the optimal solution can then we written as $\mathrm{OPT} = \sum_x w(x)^2$. The main implication of

approximation stability is that most of the points are much closer to their own center than to the centers of other clusters. Specifically:

Lemma 18. *If the instance $(\mathcal{M}, X)$ satisfies $(1+\alpha, \epsilon)$-approximation-stability then less than $6\epsilon n$ points satisfy $w_2(x)^2 - w(x)^2 \leq \frac{\alpha \text{OPT}}{2\epsilon n}$.*

Proof. Let $\mathcal{C}^*$ be the optimal k-means clustering. First notice that by approximation-stability $dist(\mathcal{C}^*, \mathcal{C}_T) = \epsilon^* \leq \epsilon$. Let B be the set of points that satisfy $w_2(x)^2 - w(x)^2 \leq \frac{\alpha \text{OPT}}{2\epsilon n}$. Let us assume that $|B| > 6\epsilon n$. We will create a new clustering $\mathcal{C}'$ by transferring some of the points in B to their second closest center. In particular it can be shown that there exists a subset of size $|B|/3$ such that for each point reassigned in this set, the distance of the clustering to $\mathcal{C}^*$ increases by $1/n$. Hence we will have a clustering $\mathcal{C}'$ which is 2ϵ away from $\mathcal{C}^*$ and at least ϵ away from $\mathcal{C}_T$. However the increase in cost in going from $\mathcal{C}^*$ to $\mathcal{C}'$ is at most αOPT. This contradicts the approximation stability assumption. □

Let us define $d_{crit} = \sqrt{\frac{\alpha \text{OPT}}{50\epsilon n}}$ as the critical distance. We call a point x *good* if it satisfies $w(x)^2 < {d_{crit}}^2$ and $w_2(x)^2 - w(x)^2 > 25{d_{crit}}^2$. Otherwise we call x as a *bad* point. Let B be the set of all bad points and let G_i be the good points in target cluster i. By Lemma 18 at most $6\epsilon n$ points satisfy $w_2(x)^2 - w(x)^2 > 25d_{crit}^2$. Also from Markov's inequality at most $\frac{50\epsilon n}{\alpha}$ points can have $w(x)^2 > {d_{crit}}^2$. Hence $|B| = O(\epsilon/\alpha)$.

Given Lemma 18, if we then define the *τ-threshold graph* $G_\tau = (S, E_\tau)$ to be the graph produced by connecting all pairs $\{x, y\} \in \binom{\mathcal{C}}{2}$ with $d(x, y) < \tau$, and consider $\tau = 2d_{crit}$ we get the following two properties:

(1) For $x, y \in {C_i}^*$ such that x and y are good points, we have $\{x, y\} \in E(G_\tau)$.

(2) For $x \in {C_i}^*$ and $y \in {C_j}^*$ such that x and y are good points, $\{x, y\} \notin E(G_\tau)$.

(3) For $x \in {C_i}^*$ and $y \in {C_j}^*$, x and y do not have any good point as a common neighbor.

Hence the threshold graph has the structure as shown in Figure 4, where each G_i is a clique representing the set of good points in cluster i. This suggests the following algorithm for k-means clustering. Notice that unlike the algorithm for ϵ-separability, the algorithm for approximation stability mentioned below needs to know the values of the stability parameters α and ϵ[4].

[4]This is specifically for the goal of finding a clustering that nearly matches an unknown target clustering, because one may not in general have a way to identify which of two proposed solutions is preferable. On the other hand, if the goal is to find a solution of low cost, then one does not need to know α or ϵ: one can just try all possible values for d_{crit} in the algorithm and take the solution of least total cost.

Algorithm k-MEANS ALGORITHM

Input: $\epsilon \leq 1$, $\alpha > 0$, k.

1. **Initialization:** Define $d_{crit} = \sqrt{\frac{\alpha \mathrm{OPT}}{50\epsilon n}}$ [a]
2. Construct the τ-threshold graph G_τ with $\tau = 2d_{crit}$.
3. **For** $j = 1$ to k do:
   Pick the vertex v_j of highest degree in G_τ.
   Remove v_j and its neighborhood from G_τ and call this cluster $C(v_j)$.
4. **Output:** the k clusters $C(v_1), \ldots, C(v_{k-1}), S - \cup_{i=1}^{k-1} C(v_i)$.

[a] For simplicity we assume here that one knows the value of OPT. If not, one can run a constant-factor approximation algorithm to produce a sufficiently good estimate.

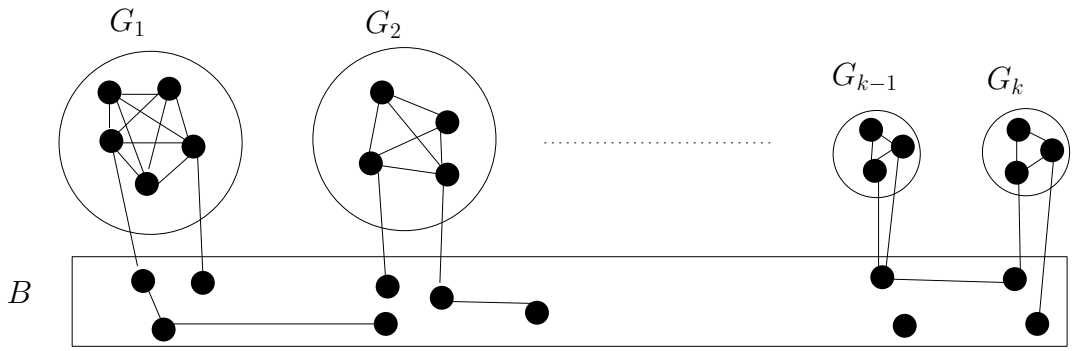

Figure 4: The structure of the threshold graph.

The authors in [20] use the properties of the threshold graph to show that the greedy method of Step 3 of the algorithm produces an accurate clustering. In particular, if the vertex v_j we pick is a good point in some cluster C_i, then we are guaranteed to extract the whole set G_i of good points in that cluster and potentially some bad points as well (see Figure 5(a)). If on the other hand the vertex v_j we pick is a bad point, then we might extract only a part of a good set G_i and miss some good points in G_i, which might lead to some errors. (Note that by property (3) we never extract parts of two different good sets G_i and G_j). However, since v_j was picked to be the vertex of the highest degree in G_τ, we are guaranteed to extract at least as many bad points as the number of missed good points in G_i see Figure 5(b). These than implies that overall we can charge the errors to the bad points, so the distance between the target clustering and the resulting clustering is $O(\epsilon/\alpha)n$, as desired.

5.6 Other notions of stability and relations between them

This notion of ϵ-separability, is in fact related to (c, ϵ)-approximation-stability. Indeed, in Theorem 5.1 of their paper, [57] show that their ϵ-separatedness assumption implies that any near-optimal solution to k-means is $O(\epsilon^2)$-close to the k-means optimal clustering. However,

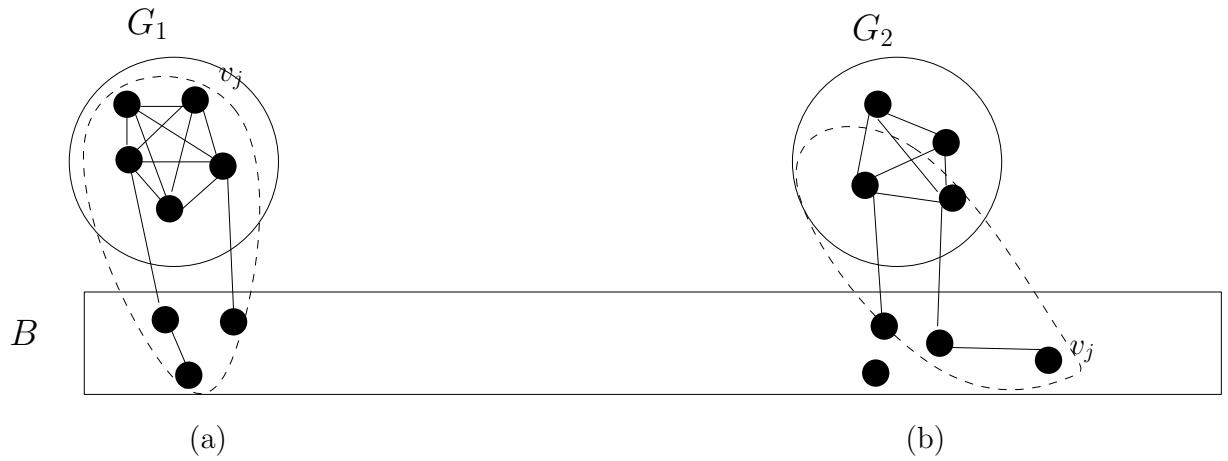

Figure 5: If the greedy algorithm chooses a good vertex v_j as in (a), we get the entire good set of points from that cluster. If v_j is a bad point as in (b), the missed good points can be charged to bad points.

the converse is not necessarily the case: an instance could satisfy approximation-stability without being ϵ-separated.[5] [21] presents a specific example of points in Euclidean space with $c = 2$. In fact, for the case that k is much larger than $1/\epsilon$, the difference between the two properties can be more substantial. See Figure 6 for an example. In addition, algorithms for approximation stability have been successfully applied in clustering problems arising in computational biology [68] (See Section 5.8 for details).

[17] study center based clustering objectives and define a notion of stability called *α-weak deletion* stability. A clustering instance is stable under this notion if in the optimal clustering merging any two clusters into one increases the cost by a multiplicative factor of $(1 + \alpha)$. This a broad notion of stability that generalizes both the ϵ-separability notion studied in section 5.1 and the approximation stability in the case of large cluster sizes. Remarkably, [17] show that for such instances of k-median and k-means one can design a $(1+\epsilon)$ approximation algorithm for any $\epsilon > 0$. This leads to immediate improvements over the works of [20] (for the case of large clusters) and of [57]. However, the runtime of the resulting algorithm depends polynomially in n and k and exponentially in the parameters $1/\alpha$ and $1/\epsilon$, so the simpler algorithms of [17] and [20] are more suitable for scenarios where one expects the stronger properties to hold. See Section 5.8 for further discussion. [3] also study various notions of clusterability of a dataset and present algorithms for such stable instances.

Kumar and Kannan [49] consider the problem of recovering a target clustering under deterministic separation conditions that are motivated by the k-means objective and by Gaussian and related mixture models. They consider the setting of points in Euclidean space, and show that if the projection of any data point onto the line joining the mean of its cluster in the target clustering to the mean of any other cluster of the target is $\Omega(k)$ standard deviations closer to its own mean than the other mean, then they can recover the target clusters in polynomial time. This condition was further analyzed and reduced by work of [18]. This separation condition is formally incomparable to approximation-stability (even restricting to the case of k-means with points in Euclidean space). In particular,

[5] [57] shows an implication in this direction (Theorem 5.2); however, this implication requires a substantially stronger condition, namely that data satisfy (c, ϵ)-approximation-stability for $c = 1/\epsilon^2$ (and that target clusters be large). In contrast, the primary interest of [21] in the case where c is below the threshold for existence of worst-case approximation algorithms.

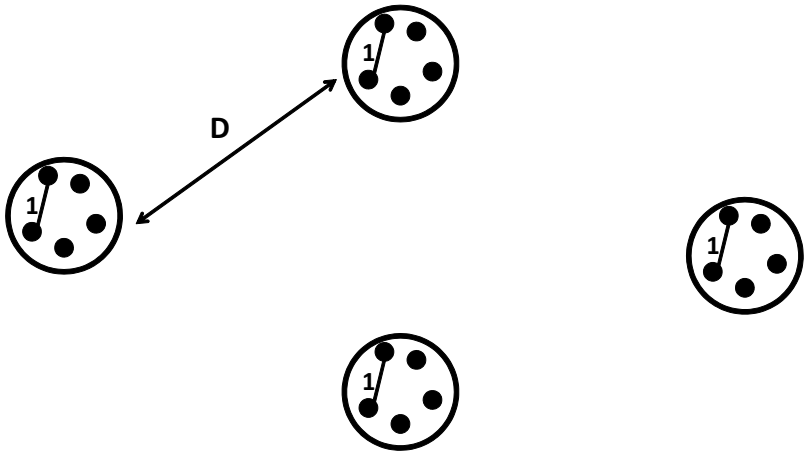

Figure 6: Suppose ϵ is a small constant, and consider a clustering instance in which the target consists of $k = \sqrt{n}$ clusters with $\sqrt{n}$ points each, such that all points in the same cluster have distance 1 and all points in different clusters have distance $D + 1$ where D is a large constant. Then, merging two clusters increases the cost additively by $\Theta(\sqrt{n})$, since D is a constant. Consequently, the optimal $(k-1)$-means/median solution is just a factor $1 + O(1/\sqrt{n})$ more expensive than the optimal k-means/median clustering. However, for D sufficiently large compared to $1/\epsilon$, this example satisfies $(2, \epsilon)$-approximation-stability or even $(1/\epsilon, \epsilon)$-approximation-stability – see [21] for formal details.

if the dimension is low and k is large compared to $1/\epsilon$, then this condition can require more separation than approximation-stability (e.g., with k well-spaced clusters of unit radius approximation-stability would require separation only $O(1/\epsilon)$ and independent of k – see [21] for an example). On the other hand if the clusters are high-dimensional, then this condition can require less separation than approximation-stability since the ratio of projected distances will be more pronounced than the ratios of distances in the original space.

Bilu and Linial [25] consider inputs satisfying the condition that the optimal solution to the objective remains optimal even after bounded perturbations to the input weight matrix. This condition is known as *perturbation resilience*. Bilu and Linial [25] give an algorithm for a different clustering objective known as maxcut. The maxcut objective asks for a 2 partitioning of a graph such the total number of edges going between the two pieces is maximized. The authors show that the maxcut objective is easy under the assumption that the optimal solution is stable to $O(n^{2/3})$-factor multiplicative perturbations to the edge weights. The work of Makarychev et al. [52] subsequently reduced the required resilience factor to $O(\sqrt{\log n})$. In [18] the authors study perturbation resilience for center-based clustering objectives such as k-median and k-means, and give an algorithm that finds the optimal solution when the input is stable to only factor-3 perturbations. This factor is improved to $1 + \sqrt{2}$ by [22], who also design algorithms under a relaxed (c, ϵ)-stability to perturbations condition in which the optimal solution need not be identical on the c-perturbed instance, but may change on an ϵ fraction of the points (in this case, the algorithms require $c = 4$). Note that for the k-median objective, (c, ϵ)-approximation-stability with respect to $\mathcal{C}^*$ implies (c, ϵ)-stability to perturbations because an optimal solution in a c-perturbed instance is guaranteed to be a c-approximation on the original instance;[6] so, (c, ϵ)-stability to per-

[6]In particular, a c-perturbed instance $\tilde{d}$ satisfies $d(x, y) \leq \tilde{d}(x, y) \leq cd(x, y)$ for all points x, y. So, using Φ to denote cost in the original instance, $\tilde{\Phi}$ to denote cost in the perturbed instance and using $\tilde{\mathcal{C}}$ to denote the optimal clustering under $\tilde{\Phi}$, we

turbations is a weaker condition. Similarly, for k-means, (c, ϵ)-stability to perturbations is implied by (c^2, ϵ)-approximation-stability. However, as noted above, the values of c known to lead to efficient clustering in the case of stability to perturbations are larger than for approximation-stability, where any constant $c > 1$ suffices.

5.7 Runtime Analysis

Below we provide the run time guarantees of the various algorithms discussed so far. While these may be improved with appropriate data structures, we assume here a straightforward implementation in which computing the distance between two data points takes time $O(d)$, as does adding or averaging two data points. For example, computing a step of Lloyd's algorithm requires assigning each of the n data points to its nearest center, which in turn requires taking the minimum of k distances per data point (so $O(nkd)$ time total), and then resetting each center to the average of all data points assigned to it (so $O(nd)$ time total). This gives Lloyd's algorithm a running time of $O(nkd)$ per iteration. The k-means++ algorithm has only a seed-selection step, which can be run in time $O(nd)$ per seed by remembering the minimum distances of each point to the previous seeds, so it has a total time of $O(nkd)$.

For the ϵ-separability algorithm, to obtain the sampling probabilities for the first two seeds one can compute all pairwise distances at cost of $O(n^2d)$. Obtaining the rest of the seeds is faster since one only needs to compute distances to previous seeds, so this takes time $O(ndk)$. Finally there is a greedy deletion procedure at time $O(ndk)$ per step for $O(k)$ steps. So the overall time is $O(n^2d + ndk^2)$.

For the approximation-stability algorithm, creating a graph of distances takes time $O(n^2d)$, after which creating the threshold graph takes time $O(n^2)$ if one knows the value of d_{crit}. For the rest of the algorithm, each step takes time $O(n)$ to find the highest-degree vertex, and then time proportional to the number of edges examined to remove the vertex and its neighbors. Over the entire remainder of the algorithm this takes time $O(n^2)$ total. If the value of d_{crit} is not known, one can try $O(n)$ values, taking the best solution. This gives an overall time of $O(n^3 + n^2d)$.

Finally, for local search, one can first create a graph of distances in time $O(n^2d)$. Each local swap step has $O(nk)$ pairs (x, y) to try, and for each pair one can compute its cost in time $O(nk)$ by computing the minimum distance of each data point to the proposed k centers. So, the algorithm can be run in time $O(n^2k^2)$ per iteration. The total number of iterations is at most $poly(n)$ [7] so the overall running time is at most $O(n^2d + n^2k^2poly(n))$. As can be seen from the table below the algorithms become more and more computationally expensive if one needs formal guarantees on a larger instance space. For example, the local search algorithm provides worst case approximation guarantees on all instances but is very slow. On the other hand Lloyd's method and k-means++ are very fast but provide bad worst case guarantees, especially when the number of clusters k is large. Algorithms based on stability notions aim to provide the best of both worlds by being fast and provably good on well behaved instances. In the conclusion section 7 we outline a guideline for practitioners when working with the various clustering assumptions.

have $\Phi(\tilde{C}) \leq \tilde{\Phi}(\tilde{\mathcal{C}}) \leq \tilde{\Phi}(\mathcal{C}^*) \leq c\Phi(\mathcal{C}^*)$.

[7] The actual number of iterations depend upon the cost of the initial solution and the stopping condition.

Method	Runtime
Lloyd's	$O(nkd)) \times (\#iterations)$
k-means++	$O(nkd)$
ϵ-separability	$O(n^2d + ndk^2)$
Approximation stability	$O(n^3 + n^2d)$
Local search	$O(n^2d + n^2k^2poly(n))$

Table 1: A run time analysis of various algorithms discussed in the chapter. The running time degrades as one requires formal guarantees on larger instance spaces.

5.8 Extensions

Variants of the k-means objective
k-means clustering is the most popular methods for vector quantization which is used in encoding speech signals and data compression [39]. There have been variants of the k-means algorithm called fuzzy kmeans which allow each point to have a degree of membership into various clusters [24]. This modified k-means objective is popular for image segmentation [1, 64]. There have also been experiments on speeding up the Lloyd's method by updating centers at each step by only choosing a random sample of the entire dataset [37]. [26] present an empirical study on the convergence properties of Lloyd's method. Rasmussen [60] contains a discussion of k-means clustering for information retrieval. [35] present an empirical comparison of k-means and spectral clustering methods. [59] study a modified k-means objective with an additional penalty for the number of clusters chosen. They motivate the new objective as a way to solve the cluster selection problem. This approach is inspired by the Bayesian model selection procedures [62]. For further details on the applications of k-means, refer to Chapters 1.2 and 2.3.

k-means++: Streaming and Parallel versions of k-means
As we saw in section 2, careful seeding is crucial in order for the Lloyd's method to succeed. One such method is proposed in the k-means++ algorithm. Using the seed centers output by k-means++ one can immediately guarantee an $O(\log k)$ approximation to the k-means objective. However k-means++ is an iterative method which needs to be repeated k times in order to get a good set of seed points. This makes it undesirable for use in applications involving massive datasets with thousands of clusters. This problem is overcome in [19] where the authors propose a scalable and parallel version of kmeans++. The new algorithm runs in much fewer iterations and chooses more than one seed point at each step. The authors experimentally demonstrate that this leads to much better computational performance in practice without losing out on the solution quality. In [6] the authors design an algorithm for k-means which makes a single pass over the data. This makes it much more suitable for applications where one needs to process data in the streaming model. The authors show that if one is allowed to store a little more than k centers ($O(k \log k)$) then one can also achieve good approximation guarantees and at the same time have an extremely efficient algorithm. They experimentally demonstrate that the proposed method is much faster than known implementations of the Lloyd's method. There has been subsequent work on improving the approximation factors and making the algorithms more practical [63].

Approximation Stability in practice
Motivated by clustering applications in computational biology, [68] analyze (c, ϵ)-approximation-stability in a model with unknown distance information where one can only make a limited number of *one versus all* queries. [68] design an algorithm that given (c, ϵ)-approx-

imation-stability for the k-median objective finds a clustering that is very close to the target by using only $O(k)$ one-versus-all queries in the large cluster case, and in addition is faster than the algorithm we present here. In particular, the algorithm for the large clusters case described in [20] (similar to the one we described in Section 5.4 for the k-means objective) can be implemented in $O(|S|^3)$ time, while the one proposed in [68] runs in time $O(|S|k(k + \log|S|))$. [68] use their algorithm to cluster biological datasets in the Pfam [38] and SCOP [56] databases, where the points are proteins and distances are inversely proportional to their sequence similarity. This setting nicely fits the one-versus all queries model because one can use a fast sequence database search program to query a sequence against an entire dataset. The Pfam [38] and SCOP [56] databases are used in biology to observe evolutionary relationships between proteins and to find close relatives of particular proteins. [68] find that for one of these sources they can obtain clusterings that almost exactly match the given classification, and for the other the performance of their algorithm is comparable to that of the best known algorithms using the full distance matrix.

6 Mixture Models

In the previous sections we saw worst case approximation algorithms for various clustering objectives. We also saw examples of how assumptions on the nature of the optimal solution can lead to much better approximation algorithms. In this section we will study a different assumption on how the data is generated in the first place. In the machine learning literature, such assumptions take the form of a probabilistic model for generating a clustering instance. The goal is to cluster correctly (with high probability) an instance generated from the particular model. The most famous and well studied example of this is the Gaussian Mixture Model (GMM)[46]. This will be the main focus of this section. We will illustrate conditions under which datasets arising from such a mixture model can be provably clustered.

Gaussian Mixture Model A univariate Gaussian random variable X, with mean μ and variance σ^2 has the density function $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{\frac{-(x-\mu)^2}{\sigma^2}}$. Similarly, a multivariate Gaussian random variable, $\mathbf{X} \in \Re^n$ has the density function

$$f(\mathbf{x}) = \frac{1}{|\Sigma|^{1/2}(2\pi)^{n/2}} e^{\left(\frac{-1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)}.$$

Here $\boldsymbol{\mu} \in \Re^n$ is called the mean vector and Σ is the $n \times n$ covariance matrix. A special case is the spherical Gaussian for which $\Sigma = \sigma^2 I_n$. Here σ^2 refers to the variance of the Gaussian in any given direction. Consider k n-dimensional Gaussian distributions, $\mathcal{N}(\boldsymbol{\mu_1}, \Sigma_1), \mathcal{N}(\boldsymbol{\mu_2}, \Sigma_2), \cdots, \mathcal{N}(\boldsymbol{\mu_k}, \Sigma_k)$. A Gaussian mixture model $\mathcal{M}$ refers to the distribution obtained from a convex combination of such Gaussian. More specifically

$$\mathcal{M} = w_1\mathcal{N}(\boldsymbol{\mu_1}, \Sigma_1) + w_2\mathcal{N}(\boldsymbol{\mu_2}, \Sigma_2) + \cdots w_k\mathcal{N}(\boldsymbol{\mu_k}, \Sigma_k).$$

Here $w_i \geq 0$, are called the mixing weights and satisfy $\sum_i w_i = 1$. One can think of a point being generated from $\mathcal{M}$ by first choosing a component Gaussian i, with probability w_i, and then generating a point from the corresponding Gaussian distribution $\mathcal{N}(\boldsymbol{\mu_i}, \Sigma_i)$. Given a data set of m points coming from such a mixture model, a fairly natural question